# UVM config_db: Passing vif and Config Through the Hierarchy

Oct 15, 2026 · @srikar

## What config_db is

`uvm_config_db` is a global lookup table. `set` writes an entry keyed by (path, field name); `get` reads it back. Nothing is assigned to a component until that component calls `get` itself.

```
uvm_config_db#(T)::set(cntxt, inst_name, field_name, value);
uvm_config_db#(T)::get(cntxt, inst_name, field_name, value);
```

The type parameter is written `#(T)`, attached directly to the class name before `::`.

| Arg | Meaning |
|---|---|
| `cntxt` | Starting component. `null` = path is absolute from the top; `this` = path is relative to the calling component. |
| `inst_name` | Hierarchical path; wildcards allowed (`"*"`, `"*agent*"`). |
| `field_name` | The lookup key. Must match the `get` string exactly, character for character. |
| `value` | The value stored (set) or the variable filled in (get). Its type must match `T`. |

`set` is the producer, `get` is the consumer. `get` returns 1 on a match and 0 otherwise, so always check it.

## How paths resolve

The effective path is `cntxt.get_full_name() + "." + inst_name`. With `null`, it is just `inst_name`. A relative path concatenates; it never adds an extra level.

| Called from | cntxt | inst_name | Effective path |
|---|---|---|---|
| tb_top | `null` | `"uvm_test_top.top_env.car_agent"` | `uvm_test_top.top_env.car_agent` |
| top_env | `this` | `"car_agent"` | `uvm_test_top.top_env.car_agent` |
| car_agent | `this` | `"*"` | `uvm_test_top.top_env.car_agent.*` |
| car_agent (get) | `this` | `""` | `uvm_test_top.top_env.car_agent` |

Path segments come from the name strings passed to `create()`, not from variable names. `create("top_env", this)` is what puts `top_env` in the path.

### Step 1: tb_top publishes the vif

The interface lives in the static module world, which classes cannot reach cleanly. `set` from tb_top is the bridge into class land.

```
// tb_top.sv
dv_car_if carif();

initial begin
  uvm_config_db#(virtual dv_car_if)::set(null, "uvm_test_top.top_env.car_agent",
"vif", carif);
  run_test();
end
```

Table entry created: (`uvm_test_top.top_env.car_agent`, `"vif"`) → `carif`.

### Step 2: env publishes the agent config

The env decides how the agent behaves (active/passive, coverage, and so on) and hands that over as a config object.

```
// dv_top_env.sv, build_phase
car_cfg   = dv_car_configuration::type_id::create("car_cfg");
car_agent = dv_car_agent::type_id::create("car_agent", this);
uvm_config_db#(dv_car_configuration)::set(this, "car_agent", "config", car_cfg);
```

Table entry created: (`uvm_test_top.top_env.car_agent`, `"config"`) → `car_cfg`. Same path as the vif entry, different key.

### Step 3: the agent gets, republishes, creates

The agent pulls both entries meant for its own path, then republishes the same values under a wider path so its children can find them.

```
// dv_car_agent.sv
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  // 1. Get what tb_top and the env published for this agent
  if (!uvm_config_db#(virtual dv_car_if)::get(this, "", "vif", vif))
    `uvm_fatal("NOVIF", "vif not found")
  if (!uvm_config_db#(dv_car_configuration)::get(this, "", "config", cfg))
    `uvm_fatal("NOCFG", "config not found")

  // 2. Republish to every child below this agent
  uvm_config_db#(virtual dv_car_if)::set(this, "*", "vif", vif);
  uvm_config_db#(dv_car_configuration)::set(this, "*", "config", cfg);

  // 3. Create children
  mon = dv_car_monitor::type_id::create("mon", this);
  if (cfg.is_active == UVM_ACTIVE) begin
    drv = dv_car_driver::type_id::create("drv", this);
    sqr = dv_car_sequencer::type_id::create("sqr", this);
  end
endfunction
```

Republishing does not change the keys or the values. The key stays `"vif"` / `"config"` and the value stays the same handle. Only the **path** changes, from `car_agent` to `car_agent.*`, so the entries now match the children.

| Entry | Path | Key | Matches |
|---|---|---|---|
| From tb_top | `uvm_test_top.top_env.car_agent` | `vif` | car_agent only |
| From top_env | `uvm_test_top.top_env.car_agent` | `config` | car_agent only |
| From car_agent | `uvm_test_top.top_env.car_agent.*` | `vif` | drv, mon, sqr and deeper |
| From car_agent | `uvm_test_top.top_env.car_agent.*` | `config` | drv, mon, sqr and deeper |

The `*` matches any descendant but not the agent itself, since the pattern requires something after the dot. The order of `set` and `create` inside the agent's build does not matter: `build_phase` runs top-down, so the children's `get` runs only after this function finishes.

To target specific children instead of all of them, name them: `set(this, "drv", "vif", vif)`.

### Step 4: driver and monitor get

Each child looks up entries for its own full path, for example `uvm_test_top.top_env.car_agent.drv`, which matches the agent's `car_agent.*` entries.

```
// dv_car_driver.sv (same in dv_car_monitor.sv)
function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  if (!uvm_config_db#(virtual dv_car_if)::get(this, "", "vif", vif))
    `uvm_fatal("NOVIF", "vif not found")
  if (!uvm_config_db#(dv_car_configuration)::get(this, "", "config", cfg))
    `uvm_fatal("NOCFG", "config not found")
endfunction
```

### Why republish to the children

The original entries match exactly one path, `uvm_test_top.top_env.car_agent`. The driver looks up `uvm_test_top.top_env.car_agent.drv`, which does not match, so without republishing its `get` returns 0 and the fatal fires.

There are three ways to get the values down:

| Approach | How | Trade-off |
|---|---|---|
| Widen the original set | tb_top uses `"uvm_test_top.top_env.car_agent*"` | tb_top and env must know the agent's internals; renames break silently |

| Approach | How | Trade-off |
|---|---|---|
| Republish from the agent | `set(this, "*", ...)` in the agent (Step 3) | Agent stays self-contained; uses string lookups |
| Assign directly | `drv.vif = vif; mon.cfg = cfg;` after `create` | Most explicit; no string-match risk |

The second and third keep each level talking only to its direct children: tb_top → test/env → agent → driver/monitor. The outside world never needs to know the agent has a `drv` or `mon`, which is what makes the agent reusable as a drop-in VIP. The agent is also the single point of control, for example creating no driver in passive mode.

A common style is config_db at the boundaries (tb_top → env → agent) and direct assignment inside the agent.

## Gotchas and debugging

- **Type must match exactly.** `set#(int)` and `get#(bit[31:0])` never match. The config object uses `#(dv_car_configuration)`, never `#(virtual dv_car_if)`.
- **Keys are exact strings.** `"config "` (trailing space) and `"config"` are different keys; the mismatch fails silently.
- **Get in `build_phase`.** Top-down build ordering guarantees the parent's `set` runs first.
- **Precedence.** During build, a `set` from higher in the hierarchy wins (the test overrides the env). After build, the last write wins.
- **Trace lookups** with `+UVM_CONFIG_DB_TRACE` on the command line, or call `uvm_top.print_config()` to dump what is set and where.